

Data Article

Inventory of Load Balancing Parameters in MPTCP Schedulers

Mohamed Rabie Naimi^{a,*}, Chakib Zouaoui^b, Mohamed Elbahri^c, Abdenacer Bounoua^b^a Djillali Liabes University of Sidi-Bel-Abbes, Telecommunications Department, Communication Networks Architecture and Multimedia Laboratory, Algeria^b Djillali Liabes University of Sidi-Bel-Abbes, Electronics Department, Communication Networks Architecture and Multimedia Laboratory, Algeria^c Djillali Liabes University of Sidi-Bel-Abbes, Computer Science Department Department, Communication Networks Architecture and Multimedia Laboratory, Algeria

ARTICLE INFO

Keywords:

Multi-path transport layer
MPTCP
Packet scheduling
decision parameter
kernel modules

ABSTRACT

In the era of ubiquitous connectivity, where multiple wired and wireless interfaces connect internet users, we can now ensure the reliability of online services. Multi-Path TCP (MPTCP) significantly improves data transport efficiency by aggregating multiple network paths into a single session to optimize network resources use. MPTCP's effectiveness depends on its scheduling decisions, which directly affect data flow, throughput, and service quality. This work presents a first labeled dataset detailing critical MPTCP scheduling parameters—congestion window sizes, unacknowledged transmitted segments, and latency—from the most popular classic schedulers: RR (RoundRobin), BLEST (Blocking Estimation-based MPTCP Scheduler), and ECF(Earliest Completion First). It allows an in-depth study of different load-balancing policies, thus clarifying the complexity of schedulers and the impact of parameters used in each load-balancing policy on data transfer. With **80033271** rows extracted across **15 scenarios**, providing usable **21** numeric values per row, the objective of this dataset is to inventory all the decisive parameters in schedulers' decision-making. This dataset will not only facilitate the selection of optimal parameters for load-balancing policies, but also serve as a foundation for the development of numerous novel, supervised machine learning methods specifically tailored for scheduling tasks.

Specifications Table

Subject	Computer Networks and Communications
Specific subject area	Inventory of parameters taken in load balancing decision policy of MPTCP schedulers
Type of data	CSV
How data were acquired	Data collected using two Linux nodes, four cisco routers, two cisco switch, and scripts to automate the data collection process.
Data format	Raw
Parameters for data collection	A network architecture was designed to control the throughput of each of the four paths for analysing the load balancing decision parameters of three schedulers (RR, Blest and Ecf) in different scenarios, using FTP as traffic generator.
Description of data collection	Server-side data collection leveraged a get_info function added to each scheduler's code. This function logged decision-making parameters via printk to the system log (/var/log/syslog), capturing all relevant load balancing variables.
Data source location	City/Town/Region: RCAM laboratory of Djillali Liabes University Country: Algeria

(continued on next column)

(continued)

Data accessibility	Repository name: Mendeley Data DOI: 10.17632/3j93c74fs7.1 Direct URL to data: https://data.mendeley.com/datasets/3j93c74fs7/1
Related research article	
Related project(s)	

Value of the Data

By providing open access to our dataset, we aim to promote collaborative research and accelerate the optimization of MPTCP schedulers.

We provide full access to the scripts, source codes, and methods to facilitate regeneration and refine the dataset for future research.

This dataset offers valuable insights into scheduling decision processes, essential for developing new, more sophisticated, and efficient load-balancing policies.

The dataset provides an opportunity to investigate the impact of various parameters on schedulers' decision-making processes.

* Corresponding author.

E-mail addresses: rabie.naimi@univ-sba.dz, naimi.m.r@gmail.com (M.R. Naimi).

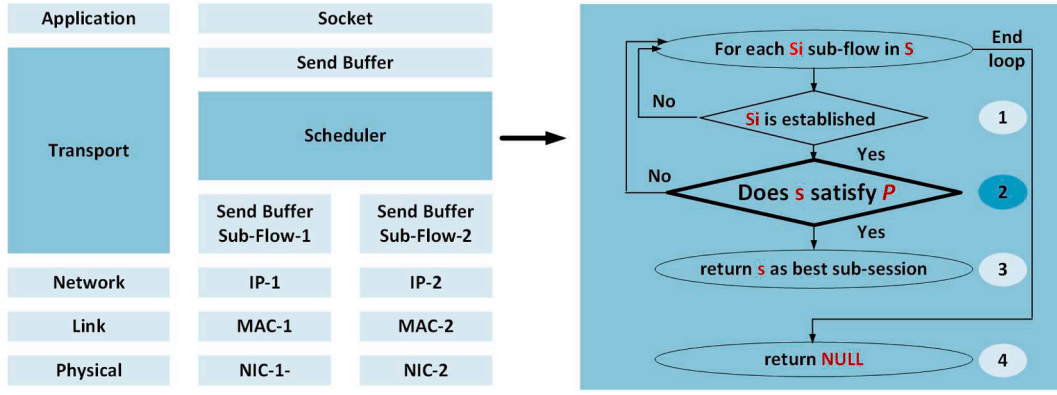


Fig. 1. MPTCP and scheduling process. (a) TCP/IP Stack. (b) MPTCP Position. (c) Scheduling Principle: It is composed of four steps, with the blue-colored step (step -2-) being the most important.

Table 1

ML Assisted schedulers

ML-Schedulers	Parameters of current sub-session	In
Reles: A neural adaptive multipath scheduler based on deep reinforcement learning	TH,CWND,RTT,FS	[6]
Peekaboo: Learning-based Multipath Scheduling for Dynamic Heterogeneous Environments	CWND,RTT,FS	[6]
An Innovative Machine Learning Approach to Improve MPTCP Performance	TH, RTT	[7]
Falcon: Fast and accurate multipath scheduling using offline and online learning	CWND,RTT,FS	[7]

Researchers can study how congestion window sizes, in-flight segments, and round-trip times among other variables affect data transmission efficiency over a multipath network.

The dataset lays the foundation for creating AI-enhanced supervised schedulers algorithms. Its wide range of parameters provides a solid foundation for offline learning methods to strengthen through online learning models, thereby improving their real-time decision-making capabilities.

1. Data

Described in [1], MPTCP is a new solution that uses multiple paths between peers simultaneously. The protocol is an optimization of the TCP protocol, providing the components necessary to establish and use multiple TCP sub-sessions across potentially multiple paths. Once MPTCP establishes sub-sessions across multiple paths, it critically schedules new segments across different sub-sessions to perform load balancing [2]. As shown in Figure Fig. 1(c), the "For each TCP sub-session in S" macro is a loop that discovers all possible TCP sub-sessions included in the overall MPTCP global session (S) between the sender and the receiver; so this process determines the number of all possible sub-sessions. However, a TCP sub-session becomes available only after establishing a three-way handshake and a non-saturated congestion window. If the sub-session is available, it goes to the second verification in step -2- Figure Fig. 1(c); otherwise, the loop verifies another sub-session. Step -2- is the most important process to generate the desired parameters of our study; it specifies that the current sub-session will be used if it satisfies the load balancing policy specific to the current scheduler.

We notice that each scheduler has its load-balancing policy based on different parameters. Our goal with this dataset is to detail the various parameters used by the different schedulers. The scheduler algorithm returns the sub-session that satisfies its policy in step -3- in Figure Fig. 1

(c). If not, the system will examine another sub-session until all potential sub-sessions have been assessed (step -4-).

The decisions made by the scheduler are crucial, as they significantly influence the multipath transmission performance parameters [3]. It is worth noting that the decision to send a segment over a sub-session is an NP-complete problem [4]. To date, no load-balancing policy is optimal, which explains researchers' choice to turn towards AI-assisted schedulers [5].

This dataset not only streamlines the selection of optimal parameters for load-balancing policies but also serves as a foundational resource for machine learning approaches, addressing the significant demand for high-quality data.

According to recent works [6,7], which survey machine learning applications in scheduling, several ML assisted schedulers (See Table 1) have been implemented with unsupervised dataset. In [8] the authors highlight the need of supervised learning techniques. It is important to note that all ML assisted schedulers are implemented without drawing on the rich experience of classic schedulers. However, a supervised dataset can serve as a robust foundation for ML schedulers. Each classic scheduler is based on its own policy's parameters, to establish the decision to use or not the current sub-session with a comparative policy between the optimal and the alternative sub-sessions.

Our dataset is mostly made with parameters used in classical schedulers: RoundRobin [2], BLEST [9], and ECF [10], along with a wider range of parameters used in other scheduler strategies. In [3], the authors list the fundamental parameters used by several classic schedulers such as CWND, number of segments in flight, and sRTT as the most frequently used ones. Since the MPTCP protocol works only between servers and clients, with networks considered as a black box, these parameters play an important role in describing the state of sub-sessions (e.g., RTT, CWND) and the state of the sending node (e.g., the buffer).

Our dataset lists all the parameters chosen for decision-making by the different schedulers: classic [2,9,10] and ML [6,7]. We directly extracted the parameter values from the kernel modules of the RoundRobin, BLEST and ECF schedulers. We separated them with commas to create a convenient CSV. The dataset's total size in CSV format is 13 Go, reflecting MPTCP traffic across 15 scenarios. The dataset comprises 80033271 rows, each containing 21 usable numeric parameters, as detailed below:

Path Identity (Pid): Uniquely identify a sub-session with a two-tuple: source IP address and destination IP address.

Window Size (CWND): This is the maximum number of segments the sender can send before requiring a cumulative acknowledgment from the receiver. All schedulers use this metric, because it plays a crucial role in determining network conditions and efficiently managing traffic flow. CWND is the only parameter in the RR scheduler [2,3].

Smoothed Round-Trip Time (sRTT): is one of the variables used to evaluate the state of the sub-session, calculated according to the RTT.

Table 2
Parameters names and value

Current Sub-session										Fast Sub-session										MPTCP Global Session										Ts	Lb	
Pid	CWND	sRTT	Th	Ds	Te	Gp	Fs	Bo	Pid	CWND	sRTT	Th	Ds	Te	Gp	Fs	Bo	Pid	CWND	sRTT	Fs	Bo			Pid	CWND	sRTT	Fs	Bo			
192.168.10.1	45	14087	36	22	1486	169	19	51452	192.168.10.1	10	10858	10	1	1107	10	3	13540	192.168.10.1	126848	9064	13	67700	926517954200	1	192.168.10.1	126848	9064	13	67700	926517954200		
+192.168.20.1									+192.168.21.1									+192.168.20.1							+192.168.20.1							
192.168.10.1	51	15656	37	27	2617	117	11	29788	192.168.10.1	11	10669	11	2	1168	19	9	29788	192.168.10.1	125184	9064	22	62056	926518021600	1	192.168.10.1	125184	9064	22	62056	926518021600		
+192.168.20.1									+192.168.21.1									+192.168.20.1							+192.168.20.1							
192.168.10.1	108	123636	9	49	15655	35	5	13540	192.168.10.1	34	75724	5	13	14839	10	34	92072	192.168.10.1	423424	9064	39	105612	926562572000	1	192.168.10.1	423424	9064	39	105612	926562572000		
+192.168.20.1									+192.168.21.1									+192.168.20.1							+192.168.20.1							
192.168.10.1	108	121563	10	40	15035	30	9	24372	192.168.10.1	14	36121	4	12	5907	23	6	20536	192.168.10.1	340608	9064	17	131564	926546689600	0	192.168.10.1	340608	9064	17	131564	926546689600		
+192.168.20.1									+192.168.21.1									+192.168.20.1							+192.168.20.1							
192.168.10.1	26	275806	1	32	37694	9	4	10832	192.168.10.1	108	24578	50	49	3245	172	37	105084	192.168.10.1	768512	9064	39	109672	926600252500	0	192.168.10.1	768512	9064	39	109672	926600252500		
+192.168.21.1									+192.168.20.1									+192.168.20.1							+192.168.20.1							

The RTT is determined by measuring the time difference between the transmission of the first segment in the congestion window and the receipt of the cumulative acknowledgment. The sRTT is a weighted average of the new RTT and the old sRTT, so it does not reflect the exact current state of the sub-session but an estimation. sRTT is used as the only parameter by the Lowest RTT First (LRTT) scheduler and acts as a base parameter for the scheduling policy of other ones [2,3].

Throughput (Th): The two key parameters determining the estimated throughput are the congestion window (CWND) and the smoothed round trip time (sRTT); it represents the number of segments (CWND) sent, but not yet acknowledged by the receiver, within a fixed time (sRTT) [7]. The HSR scheduler in [11] uses it as the only parameter for its scheduling decision.

Delivered Segments (Ds): This metric measures the number of TCP segments, which the receiver successfully acquitted within a specific time interval (RTT). While schedulers avoid using this variable, certain modern congestion control algorithms [12] employ it. We introduce this metric for potential use by future schedulers.

Time elapsed (Te): The actual duration between the sending of the first segment and the reception of the cumulative acknowledgment confirming the reception of all sent segments, thus well delivered. Similar to the Segments Delivered variable, this measure is not yet part of any scheduling algorithm [12], but we aim to integrate it for potential future use.

Goodput (Gp): is calculated using two parameters: the number of segments sent and acknowledged by the receiver (DS) in the elapsed time (TE) of transmissions. Unlike throughput, goodput reflects the actual performance of the sub-session [7,12].

In-flight segments (Fs): is the number of segments transmitted but not yet acquitted by the destination. Schedulers can monitor sub-session availability by estimating the remaining capacity within the Congestion Window (CWND) for new incoming segments [9].

Sender Buffer Occupancy (Bo): On the source side, this variable presents the amount of memory occupied by the segments waiting to be transmitted or that have been transmitted but have not yet been acquitted by the destination. This information is crucial for the ECF (Earliest Completion First) scheduler policy.

Timestamp (Ts): This variable measures the exact moment of data collection. It precisely marks the time when the scheduler makes decisions. We have included the timestamp to facilitate the potential need for analysing or graphing parameters over time.

Label (Lb): We introduced the binary variable (1, 0) as a label to facilitate the creation of a supervised model. It indicates the scheduler's decision to transmit or not a segment on a sub-session. If a segment scheduled for a sub-session, the variable label is set to '1'; if the segment is not transmitted in the same sub-session so the label is set to '0'.

BLEST [9] and ECF [10] schedulers compare the current sub-session's parameters to those of the fast sub-session and the MPTCP global session to decide if they should send a segment on this sub-session. We consolidated the variables from all three flows into a single row to streamline data analysis. To enhance visibility, we have split Table 2 into three parts, each designated for a type of sub-session: parameters of the current sub-session, parameters of the fast sub-session, and parameters of the global MPTCP session. Additionally, we included 'Timestamp' and 'Label' at the end of each row for comprehensive tracking and analysis. We provide readers with five instances as examples.

Table 2 presents some parameters values for each dataset field (5 rows). The generated dataset consists of 25 parameters in total; 21 are directly usable for training machine-learning models. Of these 21 parameters, one represents the label, while the others reflect the scheduler's metrics from the current sub-session, fast sub-session, and global session described previously. The remaining four parameters, representing IP addresses (Pid of current sub-session, fast sub-session and global session) and Timestamp are not used in metric scheduler's decision. However, we preserved them for any potential data analysis

Table 3
symbols of pseudo-code

Symbols:	
S	MPTCP Global Session
S_i	Sub-Session
S_{ic}	Current Sub-session
S_{if}	Fast Sub-Session
S_{ibest}	Best Sub-Session
$sRTT$	smoothed Round Trip Time
MSS	Maximum Segment Size
$CWND$	Congestion Window
F_s	In-flight segments
Y	Avail space in the Congestion Window of MPTCP Global Session
X	number of segments that can be transmitted on S_{if} durring $S_{ic} \rightarrow sRTT$
λ	correction factor
K	number of segments that can be transmitted on S_{if}
δ	standard deviations of $S_{if} \rightarrow sRTT$ and $S_{ic} \rightarrow sRTT$
β	correction factor

Algorithm 1
blest_get_subflow

Input: mptcp global session	
Output: best sub-session	
1	Begin
2	for each $S_i \in S$ do
3	if S_i is not established then continue
4	if $S_i \rightarrow sRTT < S_{if} \rightarrow sRTT$ then $S_{if} = S_i$
5	end for
6	$S_{ic} = \text{lr_get_subflow}()$
7	if $S_{if} \neq S_{ic}$ then
8	$rtts = S_{ic} \rightarrow sRTT / S_{if} \rightarrow sRTT$
9	$X = S_{if} \rightarrow MSS \times \left(S_{if} \rightarrow CWND + \frac{rtts - 1}{2} \right) \times S_{ic} \rightarrow CWND$
10	$Y = S_{ic} \rightarrow CWND - S_{ic} \rightarrow MSS \times (S_{ic} \rightarrow F_s + 1)$
11	if $X \times \lambda < Y$ then
12+	Send_Info(with class=0)
13	Return NULL (no available subflow)
14	end if
15	end if
16	if S_{ic} then $S_{ibest} = S_{ic}$
17+	Send_Info(with class=1)
18	return S_{ibest}
19	End

process.

To facilitate quick access and familiarity with the dataset, we have saved it in CSV format and compressed it into a multiple file. The dataset is organized according to different schedulers, congestion algorithms, and scenarios. The compressed folders are named following this convention: <scenario>_<name of scheduler>_< name of congestion control algorithm>.csv. The following section provides details on generation of our dataset.

2. Experimental Design, Materials, and Methods

The generation of our dataset followed a three-phase process. In the first phase, we studied the scheduling process in-depth and the parameters used by BLEST, ECF and RR in their load-balancing policies. In the second phase, we examined the source codes of each scheduler to extract all invoked parameter values in the load-balancing policy. Finally, we designed a platform capable of deploying schedulers in various scenarios, with four communication paths between the server and clients. Each path has configurable throughput in symmetric and asymmetric scenarios. Using this methodology, our dataset provides valuable insights into MPTCP parameters under various network conditions.

2.1. Experimental Design

For the proposed dataset, we have selected the latest version of

Algorithm 2
ecf_get_subflow

Input: mptcp global session	
Output: best sub-session	
1	Begin
2	for each $S_i \in S$ do
3	if S_i is not established then continue
4	if $S_i \rightarrow sRTT < S_{if} \rightarrow sRTT$ then $S_{if} = S_i$
5	end for
6	$S_{ic} = \text{lr_get_subflow}()$
7	if $S_{if} \neq S_{ic}$ then
8	$n = 1 + \frac{K}{S_{if} \rightarrow CWND}$
9	$\delta = \max(\delta S_{if}, \delta S_{ic})$
10	if $n \times S_{if} \rightarrow sRTT < (1 + \text{waiting} \times \beta) (S_{ic} \rightarrow sRTT + \delta)$ then
11	if $\frac{K}{S_{ic} \rightarrow CWND} \times S_{ic} \rightarrow sRTT \geq 2 \times S_{ic} \rightarrow sRTT + \delta$ then
12	waiting = 1
13+	Send_Info(with class=0)
14	Return NULL (no available subflow)
15	end if
16	else waiting = 0
17	end if
18	end if
19	if S_{ic} then $S_{ibest} = S_{ic}$
20+	Send_Info(with class=1)
21	return S_{ibest}
22	End

Algorithm 3
rr_get_subflow

Input: mptcp global session	
Output: best sub-session	
1	Begin
2	for each $S_i \in S$ do
3	if S_i is not established then continue
4	$S_{ic} = S_i$
5	if $S_i \rightarrow sRTT < S_{if} \rightarrow sRTT$ then $S_{if} = S_i$
6	end for
7	if S_{ic} then $S_{ibest} = S_{ic}$
8+	Send_Info(with class=1)
9	return S_{ibest}
10	End

MPTCP protocol implemented on kernel 5.4 accessible at URL [13]. This kernel includes the source codes of schedulers, used in our dataset conception labelling step, detailed in this section.

To ensure clarity, Table 3 provides a comprehensive description of each symbol, along with its corresponding meaning and role within the pseudo-code of the MPTCP scheduling mechanism.

BLEST: is a state-based scheduler developed for asymmetric paths, it aims to maximize the use of fast sub-sessions with the minimum RTT. Included in the official kernel version 0.96 [13], it does not strictly adhere to the version developed by the original author [9] in version 0.86. Algorithm 1 illustrates the version integrated into the official kernel [14], which is essentially optimizing the default scheduler Low RTT (LRTT) [2]. The BLEST scheduler only use the sub-session suggested by LRTT (line 6), considered as a current sub-session in this paper. The current sub-session is utilized if it meets BLEST's specific criteria (line 11); we assign a value of 1 to our label (line 17). Otherwise, if the sub-session is not utilized, we assign a value of 0 to our label (line 12).

ECF: is also a state-based scheduler and is part of the same family as BLEST, but it considers different parameters such as send buffer size. ECF was recently integrated into version 0.96 [13], and enhanced to the use of multiple streams. These changes allow it to outperform the initially designed two-stream model. The integrated ECF into 0.96 version is an enhancement of the LRTT scheduler [13], as illustrated in Algorithm 2. We assign the binary value of labels following the decision-making process, with '0' (line 13) and '1' (line 20) representing

the use or not of the LRTT-recommended sub-session.

RoundRobin (RR): The last used scheduler is stateless; it loops circularly through all possible sub-sessions, ignoring their state [2]. Unlike ECF and BLEST, RR does not use any metric to determine whether to use a sub-session or not, as illustrated in Algorithm 3. This behaviour explains why we set the label to 1 in all lines (line 8). RR is particularly effective in symmetric scenarios, that why we have included it in our dataset as a reference scheduler, especially to observe its unique behaviour in symmetric scenarios compared to other schedulers like ECF and BLEST.

2.2. Method for Collecting Data

The protocol MPTCP is deployed on both the server and the client node, while scheduling decision is made only in the sender-side node, to balance the load between different sub-sessions. According to this, we used server side to collect data in our study. To closely replicate real network conditions, we installed the vsFTPD package on the server, to use it as a traffic generator like in [15].

We developed a void function named `send_info` in the three kernel modules (ECF, BLEST and RR); this function takes as input: the current sub-session, fast sub-session, MPTCP global session, and our label. In this function, we call the `printf` function to write in one row all the variables in the syslog file (`/var/log/syslog`). To ensure data integrity and avoid errors, all null variables have been initialized to 0. The location of `send_info` call depends on whether the sub-session is used or not for segment transmission. `send_info` retrieves and writes parameters value of the current sub-session, the fast sub-session, and the MPTCP global session, by setting the “label” variable to its appropriate value according to the each scheduler policy described in the experimental design section. This methodology ensures that dataset contains the targeted parameters of each sub-session. The three modified source codes are accessible from the URL [17].

A Python script entirely based on executing system commands was developed to facilitate the reproduction of the dataset. Accessible via the URL [17], this code is sufficient for installing all requirements to reproduce the dataset. We designed it to eliminate technical challenges and fully automate deploying and configuring the test platform, as detailed in Section 2.2.1. We outline this script into four essential steps:

- **Installing Tools and Dependencies:** This initial step orchestrates a sequence of automated actions to configure the system to support the MPTCP protocol, leveraging direct system commands. It ensures the installation of essential services such as SSH and FTP, and the deployment of the necessary tools for compiling and developing kernel modules. Additionally, the Paramiko integration lays the foundation for future use of Python scripts capable of automatically running SSH tasks on all nodes. To improve access and execution of commands via SSH, we change the root password and create a specific user named “mptcp”. This user becomes the repository of all files required to generate the dataset.
- **Downloading and Installing MPTCP Kernel v0.96:** The MPTCP v0.96 core can be download from GitHub in [16]. After downloading, the kernel installation takes place automatically. This procedure automates the integration of the MPTCP v0.96 kernel to exploit the advantages of the fully MPTCP v1 protocol.
- **Configuring SSH, VSFTPD, and GRUB Services:** Step 3 focuses on configuring SSH, vsFTPD, and GRUB services. Allowing root access via SSH is essential to manage remote nodes, especially when running automation scripts that require elevated privileges. This step also adjusts the vsFTPD service to allow writing in the receiver node, simplifying the use of an automation script to upload a file named “5Gbits” from the sender node. It is important to note that the system now hosts two separate kernels: the standard Ubuntu kernel and another one specially compiled for MPTCP. To ensure that the system

Table 4

Material descriptions

Equipments	Description
Server	Dell intel i7 g12, RAM 64 Go, 2 GigaEthern interfaces, Ubuntu 20.04 kernel v:5.4 mptcp v:0.96
Client	Dell intel i7 g12, RAM 64 Go, 2 GigaEthern interfaces, Ubuntu 20.04 kernel v:5.4 mptcp v:0.96
Tow Routers	Routers CISCO 2811, 3 GigaEthern interfaces ios version: 15.1(4)M4
Tow Routers	Routers CISCO 2821, 3 FastEthern interfaces ios version: 15.1(4)M4
Tow Switch	Switch CISCO : catalyst 2950, 48 fastEthern, 2 GigaEthern Interfaces, ios version: 12.1(22)EA14

boots with the MPTCP kernel, it must be manually selected via the GRUB menu after reboot.

Downloading and Compiling Source Codes (RR, Ecf, and Blest): The last step consists of downloading and compiling the source codes of the scheduling algorithms that we have customized, accessible from [17]. For the standard congestion control algorithms (Linked Increase Adaptation-lia, Opportunistic Linked-Increase Algorithm-olia, Balanced Linked Adaptation-balia) available on the official GitHub repository of the MPTCP project, accessible via the URL [13]. The script compile each algorithm in a separate file to streamline the automation of loading MPTCP protocol scheduling modules.

2.3. Network Architecture and Scenarios

Setting up the hardware and the software environment was crucial in our project execution. We conducted the experiments on a physical hardware platform, which include a server, client, four routers, and a Tow switch without using an emulator or simulator. Table 4 details the physical specifications of each network component.

The network architecture in figure Fig. 2, heavily inspired by [15], allows for up to four paths between a server and a client. Two separate interfaces, S1 and S2, each belonging to a different network, connect the server, while C1 and C2 connect the clients similarly.

We configured simple static routing on the routers to route segments from each server interfaces to the two client interfaces. Using the PATH MANAGER, the MPTCP protocol can exploit the following four possible paths: Path1: S1-C1, Path2: S1-C2, Path3: S2-C1, and Path4: S2-C2. To facilitate the reproduction of the architecture, configuration files are available in [17].

As part of our study on the decision-making processes of MPTCP schedulers, we conducted different test scenarios to analyse the behaviour of RR, ECF and BLEST. The objective is to observe the impact of these scenarios on the scheduler parameters. Under different conditions, we vary the number of communication paths and throughputs on each path: for example, we select paths 1 and 2 in the two-path scenarios, or we select all four paths in the second case; and for every selection, we vary the throughputs; Table 5 provides detailed descriptions of these scenarios.

To analyse the impact of congestion algorithms on scheduling performance, we tested each scheduler (RR, BLEST and ECF) in combination with different congestion control algorithms (LIA, OLIA and BALIA). This resulted in nine unique combinations: RR & LIA, RR & OLIA, RR & BALIA, BLEST & LIA, BLEST & OLIA, BLEST & BALIA, ECF & LIA, ECF & OLIA, and ECF & BALIA. For each combination, we conducted one run across 15 different scenarios, performing 135 runs (9 combinations × 15 scenarios). All runs were recorded in our dataset, the scenarios are described below:

- The first scenario is a reference because it has only one active path at 100 Mbps.

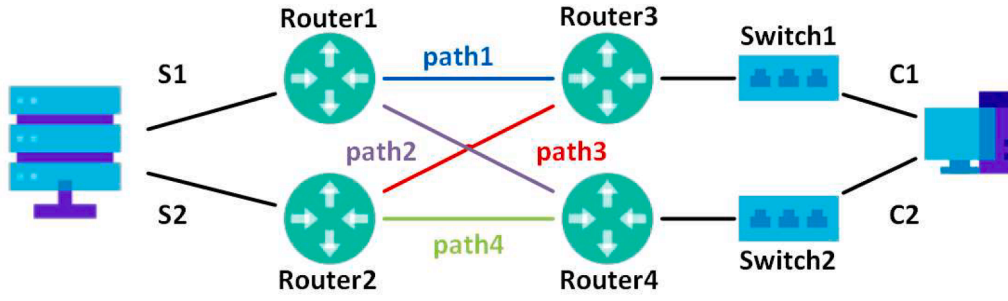


Fig. 2. network architecture

Table 5
Scenarios

Sc	Paths			
	Path 1	Path 2	Path 3	Path 4
1	100	//	//	//
2	100	100	//	//
3	100	90	//	//
4	100	80	//	//
5	100	70	//	//
6	100	60	//	//
7	100	50	//	//
8	100	40	//	//
9	100	30	//	//
10	100	20	//	//
11	100	10	//	//
12	100	100	100	100
13	100	85	75	50
14	100	75	50	25
15	100	65	30	5

- Scenarios (2, 12) describe configurations with homogeneous paths, with symmetrical sub-session rates on two and four paths. We need to perform these tests to check the behaviour of BLEST and ECF in symmetric scenarios, even though they are designed for asymmetric networks.
- In scenarios (3-11), the second path throughput gradually decreases from 90 to 10 Mbps. This makes it possible to evaluate the adaptability of schedulers to an increasingly marked throughput asymmetry between paths.

- Scenarios 13-15 are the most complex, as they involve different throughputs for a maximum number of paths. These paths represent various conditions, and the goal is to observe how schedulers handle increasingly unequal paths.

Scenarios are configured with a Python script named `mptcpScenario.py` accessible in [17] to manage paths throughputs, configurations of schedulers & congestion control algorithms and FTP requests. To configure paths throughputs, we use traffic shaping as a technique on routers. Here is a detailed explanation of the different steps and their roles:

- Configuring the throughputs of paths on the routers: This step consists of configuring the throughput of each path according to the scenario table. We have used the traffic shaping technique to regulate and control the flow of data on different paths, ensuring that the flow rates strictly corresponded to the parameters defined for each scenario.
- Configuring Schedulers and Congestion Control Algorithms: we have compiled the required modules of schedulers and congestion control with the script `mptcpInstall.py` detailed in section 1. Next, we load the desired module from the user space MPTCP into the system kernel and configure it as a scheduler or congestion control.
- Traffic Initiation and Log Retrieval: This phase aims to capture the log entries generated by the schedulers (see Section 2.2) into syslog. This phase begins by resetting the system logs at the start of each test scenario, ensuring that all data collected is specific to the current test and not noised by previous information. The script then starts a transfer via an FTP request of file named '5Gbits', designating the sending node as the server. After the file transfer, the script `mptcpSc`

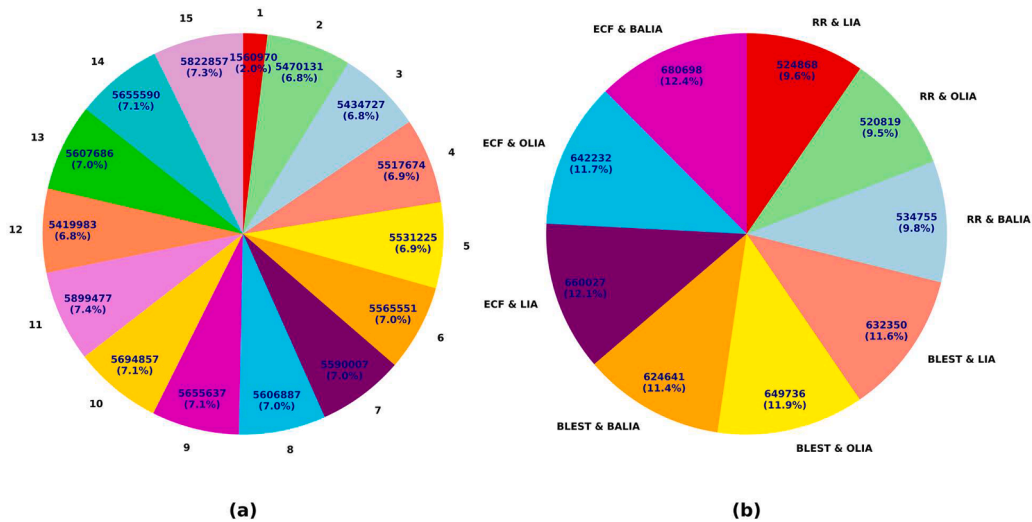


Fig. 3. Inter- and Intra-Scenario Dataset Diversity. (a) Illustrates Inter-Scenario diversity across scenarios 1 to 15, and (b) Illustrates Intra-Scenario diversity of Scenario 2.

`enario.py` extracts the parameters written into the file `syslog`. The script then saves these entries in CSV format, following the naming convention discussed earlier in the article.

We design each step to apply the right configuration dictated by scenarios, facilitating the reproduction of dataset and future comparative analysis of various MPTCP schedulers' parameters. The scenarios are range from stable single-path to more complex multipath network conditions.

Figure Fig 3 confirms the diversity of our dataset, for both inter-scenario and intra-scenario. Figure Fig 3.a illustrates the percentage distribution and number of rows across scenarios 1 to 15. Figure Fig 3.b illustrates the percentage distribution and number of rows for the nine combinations of schedulers and congestion control algorithms in scenario 2, which is used as an example. As previously mentioned, scenario 1 serves as a reference due to its single active path, resulting in a smaller size compared to the other scenarios. Excluding scenario 1, we obtained an average of 5,605,164 rows per scenario.

CRedit authorship contribution statement

Mohamed Rabie Naimi: Writing – original draft, Software, Investigation, Formal analysis, Data curation, Conceptualization. **Chakib Zouaoui:** Supervision, Software, Project administration. **Mohamed Elbahri:** Writing – original draft, Validation, Software, Investigation. **Abdenacer Bounoua:** Writing – review & editing, Validation, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

our dataset is in Mendeley Dataopen access at <https://data.mendeley.com/datasets/3j93c74fs7/1>.

Acknowledgments

The research being reported in this publication was the Algerian Directorate General for Scientific Research and Technological

Development (DGRSDT).

References

- [1] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, C. Paasch, TCP Extensions for Multipath Operation with Multiple Addresses, RFC Editor, 8684, RFC, 2020. March [Online]. Available, <https://www.rfc-editor.org/rfc/rfc8684.txt>.
- [2] C. Paasch, S. Ferlin, O. Alay, O. Bonaventure, Experimental evaluation of multipath TCP schedulers, in: Proceedings of the 2014 ACM SIGCOMM Workshop on Capacity Sharing Workshop, ACM, Chicago Illinois USA, 2014, pp. 27–32, <https://doi.org/10.1145/2630088.2631977>.
- [3] B.Y.L. Kimura, D.C.S.F. Lima, A.A.F. Loureiro, Packet Scheduling in Multipath TCP: Fundamentals, Lessons, and Opportunities, IEEE Systems Journal 15 (2021) 1445–1457, <https://doi.org/10.1109/JSYST.2020.2965471>.
- [4] J.P. Sheu, L.W. Liu, R.B. Jagadeesha, Y.C. Chang, « An efficient multipath routing algorithm for multipath TCP in software-defined networks », in: 2016 European conference on networks and communications (EuCNC), IEEE, 2016, pp. 371–376, <https://doi.org/10.1109/EuCNC.2016.7561065>.
- [5] Y. Xing, K. Xue, Y. Zhang, J. Han, J. Li, D.S.L. Wei, An Online Learning Assisted Packet Scheduler for MPTCP in Mobile Networks, IEEE/ACM Trans. Networking (2023) 1–16, <https://doi.org/10.1109/TNET.2023.3246168>.
- [6] M.Y. Asiri, A Survey of Multipath TCP Scheduling Schemes: Open Challenges and Potential Enablers, 2021. <https://doi.org/10.36227/techrxiv.14659350.v1>.
- [7] M. Maliha, G. Habibi, M. Atiquzzaman, A Survey on Congestion Control and Scheduling for Multipath TCP: Machine Learning vs Classical Approaches, (2023). <https://arxiv.org/abs/2309.09372> (accessed April 28, 2024).
- [8] R. Poorzare, H. Asghari, O.P. Waldhorst, Steps Toward a Supervised Machine Learning Scheduler for MPTCP, in: International Conference on Networked Systems & ZdN, 2023 (n.d.).
- [9] S. Ferlin, O. Alay, O. Mehani, R. Boreli, BLEST: Blocking estimation-based MPTCP scheduler for heterogeneous networks, in: 2016 IFIP Networking Conference (IFIP Networking) and Workshops, IEEE, Vienna, Austria, 2016, pp. 431–439, <https://doi.org/10.1109/IFIPNetworking.2016.7497206>.
- [10] Y.S. Lim, E.M. Nahum, D. Towsley, R.J. Gibbens, Ecf: An MPTCP path scheduler to manage heterogeneous paths, in: Proceedings of the 13th international conference on emerging networking experiments and technologies, 2017, pp. 147–159, <https://doi.org/10.1145/3143361.3143376>.
- [11] B.Y.L. Kimura, D.C.S.F. Lima, A.A.F. Loureiro, Alternative Scheduling Decisions for Multipath TCP, IEEE Commun. Lett. 21 (2017) 2412–2415, <https://doi.org/10.1109/LCOMM.2017.2740918>.
- [12] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, G. Carle, « Towards a deeper understanding of TCP BBR congestion control », in: 2018 IFIP networking conference (IFIP networking) and workshops, IEEE, 2018, pp. 1–9, <https://doi.org/10.23919/IFIPNetworking.2018.8696830>.
- [13] Linux Kernel implementation of MultiPath TCP, UCLouvain - IP Networking Lab, <https://github.com/multipath-tcp/mptcp>, 2024 (accessed 04 September 2024).
- [14] H. Cech, Technical Report, Technical University of Munich, 2020.
- [15] B. Almási, S. Szilágyi, Multipath ftp and stream transmission analysis using the MPT software environment, International Journal of Advanced Research in Computer and Communication Engineering 2 (11) (2013) 4267–4272.
- [16] Linux Kernel implementation of MultiPath TCP, UCLouvain - IP Networking Lab, <https://github.com/multipath-tcp/mptcp/releases>, 2024 (accessed 04 September 2024).
- [17] Scripts and codes for generation of MPTCP DATASET, UDL, RCAM Lab, 2024. <https://github.com/dlnmr/MptcpDataset> (accessed 04 September 2024).